SCR2043 OPERATING SYSTEMS

Name	: Nuraisyah Binti Mohd Zikre	Marks
Student ID	: A23CS0160	
Section	: 03	

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. **Download necessary source codes:**

   Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. **Compile the source files:**

   Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. **Execute the dummy processes:**

   Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

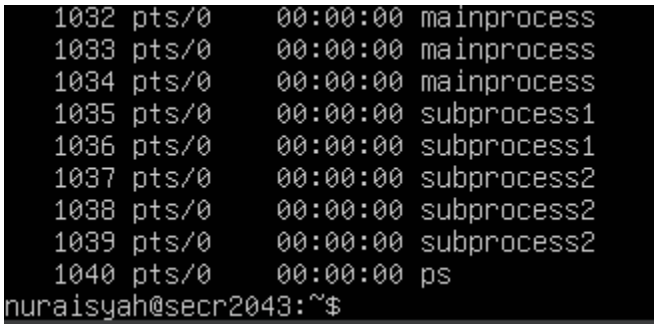
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

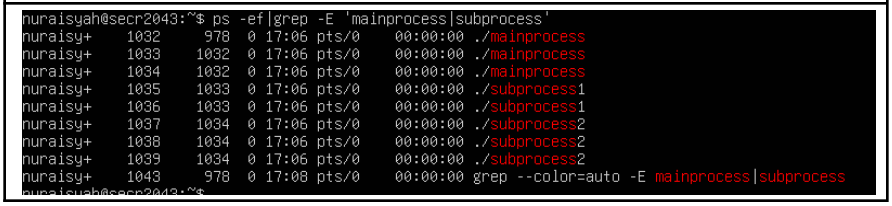
Command
`ps -e`
Output


Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
`ps -ef|grep -E 'mainprocess|subprocess'`

Output

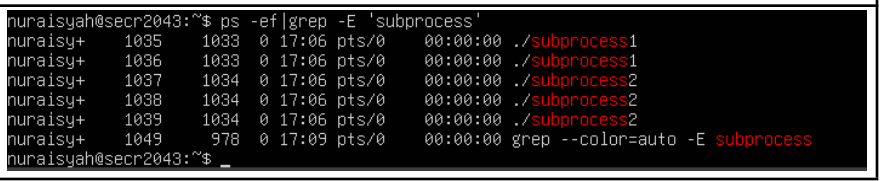


Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
`ps -ef|grep -E 'subprocess'`

Output



Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
`ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'`
Output

```
nuraisyah@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'
   1035 nuraisy+  0.0  0.1 ./subprocess1
   1036 nuraisy+  0.0  0.1 ./subprocess1
   1037 nuraisy+  0.0  0.1 ./subprocess2
   1038 nuraisy+  0.0  0.1 ./subprocess2
   1039 nuraisy+  0.0  0.1 ./subprocess2
   1055 nuraisy+  0.0  0.1 grep --color=auto -E subprocess
nuraisyah@secr2043:~$
```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
`ps -eo pid,user,pcpu,pmem,cmd --sort=pmem | grep -E 'subprocess'`
Output

```
nuraisyah@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem | grep -E 'subprocess'
   1035 nuraisy+  0.0  0.1 ./subprocess1
   1036 nuraisy+  0.0  0.1 ./subprocess1
   1037 nuraisy+  0.0  0.1 ./subprocess2
   1038 nuraisy+  0.0  0.1 ./subprocess2
   1039 nuraisy+  0.0  0.1 ./subprocess2
   1063 nuraisy+  0.0  0.1 grep --color=auto -E subprocess
nuraisyah@secr2043:~$
```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
`ps --forest -C mainprocess -C subprocess1 -C subprocess2`

Output

```
nuraisyah@secr2043:~$ ps --forest -C mainprocess -C subprocess1 -C subprocess2
    PID TTY          TIME CMD
   1032 pts/0    00:00:00 mainprocess
   1033 pts/0    00:00:00  \_ mainprocess
   1035 pts/0    00:00:00  |   \_ subprocess1
   1036 pts/0    00:00:00  |   \_ subprocess1
   1034 pts/0    00:00:00  \_ mainprocess
   1037 pts/0    00:00:00      \_ subprocess2
   1038 pts/0    00:00:00      \_ subprocess2
   1039 pts/0    00:00:00      \_ subprocess2
nuraisyah@secr2043:~$
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

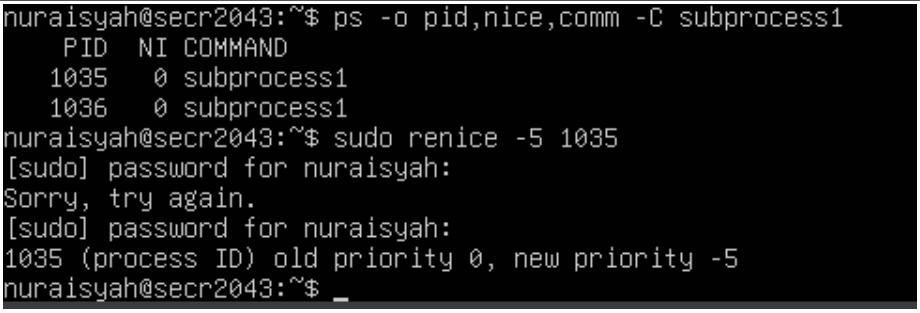
Command
`pstree -c -s 1032`

Output

```
nuraisyah@secr2043:~$ pstree -c -s 1032
systemd---sshd---sshd---sshd---bash---mainprocess-+-mainprocess-+-subprocess1
                                                  |             `-subprocess1
                                                  `-mainprocess-+-subprocess2
                                                                |-subprocess2
                                                                `-subprocess2
nuraisyah@secr2043:~$ _
```

Question 8

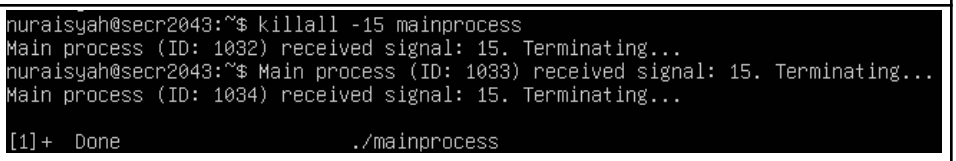
Use `renice` command to change priority level of one of process “subprocess1”.

Command
`ps -o pid,nice,comm -C subprocess1` `sudo renice -5 1035`
Output


```
nuraisyah@secr2043:~$ ps -o pid,nice,comm -C subprocess1
    PID  NI COMMAND
   1035   0 subprocess1
   1036   0 subprocess1
nuraisyah@secr2043:~$ sudo renice -5 1035
[sudo] password for nuraisyah:
Sorry, try again.
[sudo] password for nuraisyah:
1035 (process ID) old priority 0, new priority -5
nuraisyah@secr2043:~$ _
```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
`killall -15 mainprocess`
Output


```
nuraisyah@secr2043:~$ killall -15 mainprocess
Main process (ID: 1032) received signal: 15. Terminating...
nuraisyah@secr2043:~$ Main process (ID: 1033) received signal: 15. Terminating...
Main process (ID: 1034) received signal: 15. Terminating...

[1]+  Done                    ./mainprocess
```

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

Linux command:

```
nano question10.c
gcc question10.c -o question10
./question10
```

Source code questio10.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
   pid_t pid = fork();

   if (pid == 0)
   {
      printf("Child process with PID = %d\n", getpid());
   }

   else if (pid > 0)
   {
      wait(NULL);
      printf("Parent process with PID = %d\n", getpid());
   }

   else
   {
      printf("Fork failed!\n");
   }

   return 0;
}
```

Output:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
        pid_t pid = fork();

        if (pid == 0)
        {
                printf("Child process with PID: %d\n", getpid());
        }

        else if (pid > 0)
        {
                wait(NULL);
                printf("Parent process with PID: %d\n", getpid());
        }

        else
        {
                printf("Fork failed!\n");
        }

        return 0;

}
```

Output of questio10.c:

```
nuraisyah@secr2043:~$ gcc question10.c -o question10
nuraisyah@secr2043:~$ ./question10
Child process with PID: 1580
Parent process with PID: 1579
```